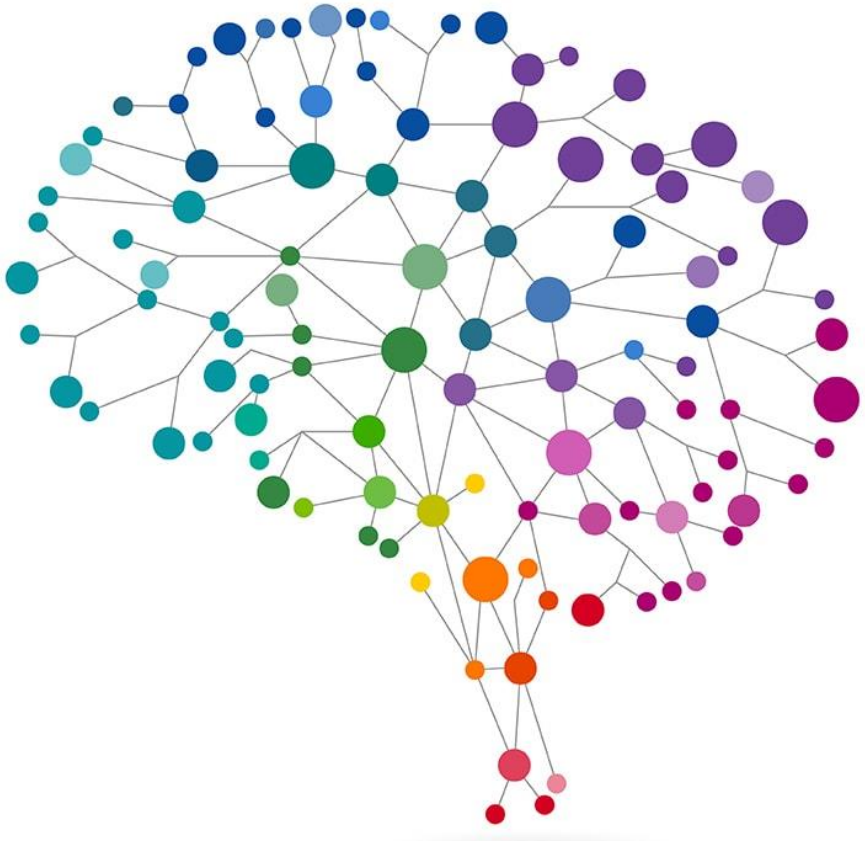
Aprendizado de Máquina





Classroom code tg44fjf

Prof. Rodrigo Caropreso
Nov/2018
rcaropreso @ gmail

Conteúdo Programático – Aulas 1 e 2

Introdução

- Conceitos Gerais em Aprendizado de Máquina
- Tipos de Aprendizado: Supervisionado, Semi-Supervisionado, Não-Supervisionado;
- Aprendizado por Reforço.

Computação Evolutiva e Inteligência Coletiva

- Otimização por Enxame de Partículas (PSO);
- Algoritmos Genéticos.

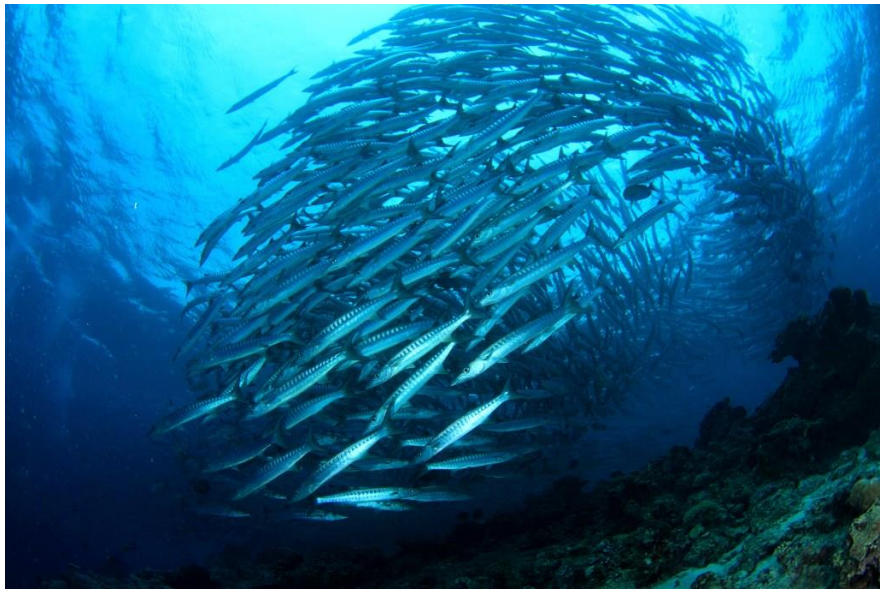
Redes Neurais Artificiais – Parte 1

- Algoritmo de Descida de Gradiente
- Perceptron Simples, Adaline;
- Lei de Aprendizado de Hebb;
- Regra Delta.



Particle Swarm Optimization (PSO)

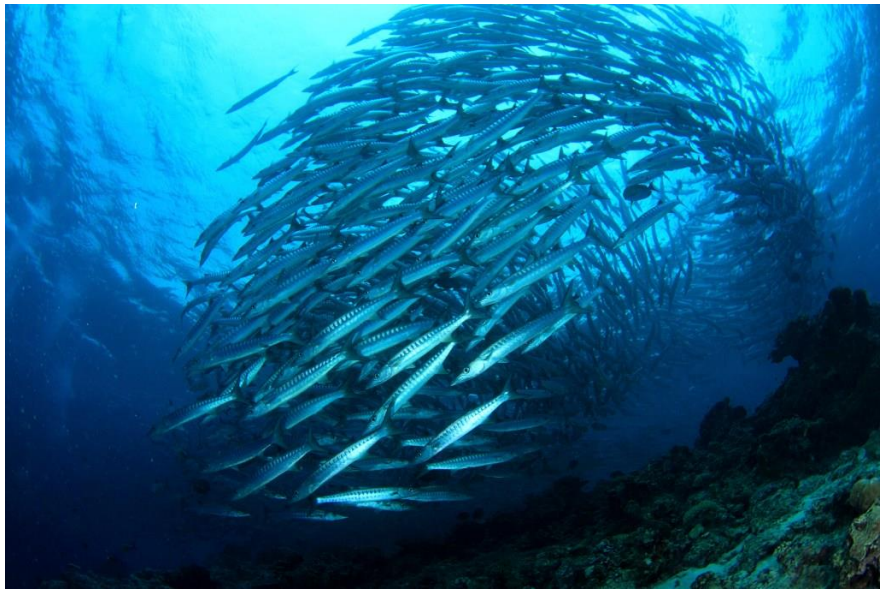
Particle Swarm Optimization: Conceitos



- Faz parte da área conhecida como **Inteligência Coletiva (Swarm),** que consiste em um conjunto de modelos computacionais que tentam imitar aquela inteligência que está distribuída quando se considera o todo, que é coordenada em tempo real, e que resulta em mobilização efetiva de competências bem simples.

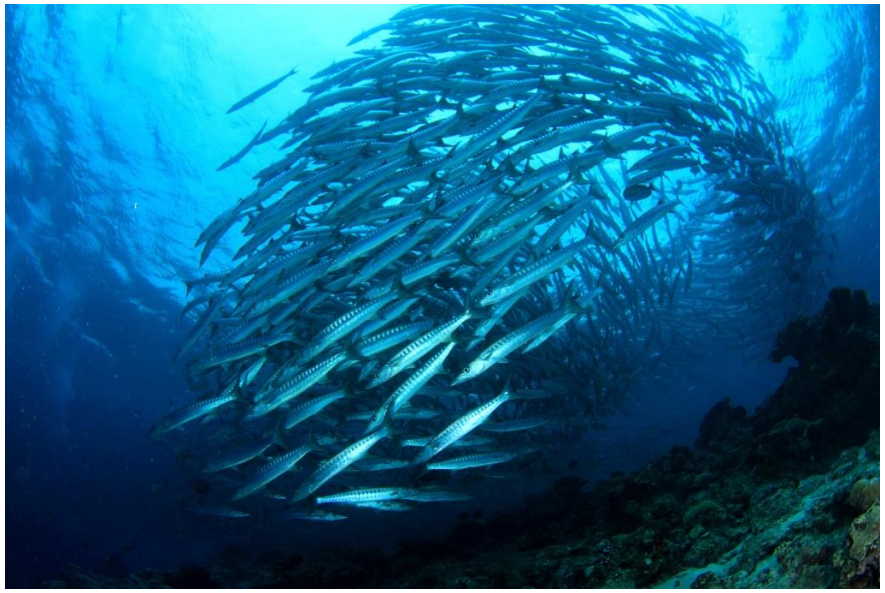
- Aplica-se normalmente em problemas relacionados à otimização de sistemas, em problemas de difícil manipulação pelas técnicas tradicionais.

Particle Swarm Optimization: Conceitos



- Os sistemas de Inteligência Coletiva são compostos tipicamente por:
  - Uma população de organismos que interagem localmente um com o outro e com seu ambiente.
  - Os agentes seguem regras bem simples, sendo que nenhuma estrutura centralizada de controle rege como os agentes (organismos) individuais devem se comportar.
  - Essas interações locais entre tais agentes conduzem à resolução de problemas com comportamento global complexo

Particle Swarm Optimization: Inteligência Coletiva



- São notáveis a interação social e a capacidade de adaptação de vários organismos. Segundo a ótica da inteligência artificial e da psicologia social:
  - Inteligência e sucesso, em muitos casos, provêm muito mais da sociabilidade do que da capacidade individual.
- A inteligência artificial distribuída é uma área de pesquisa que aborda fenômenos ligados a “inteligência social”:
  - Formigas;
  - Cupins;
  - Abelhas;
  - Peixes;
  - Pássaros.

Particle Swarm Optimization: Inteligência Coletiva



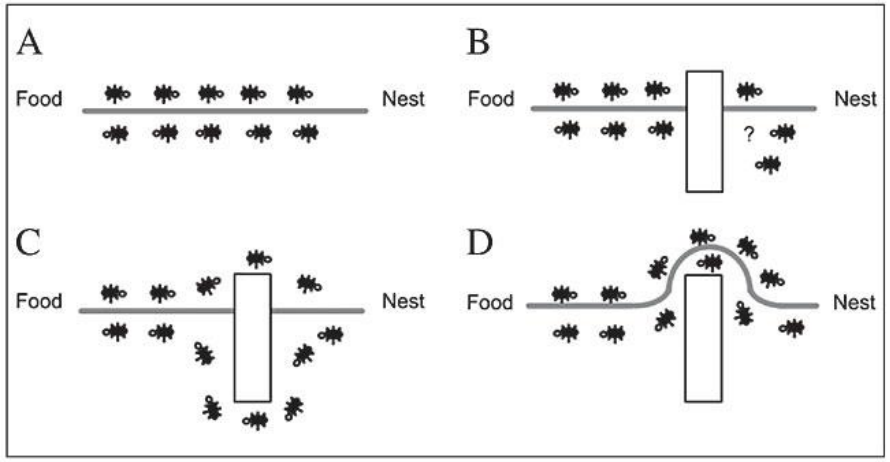
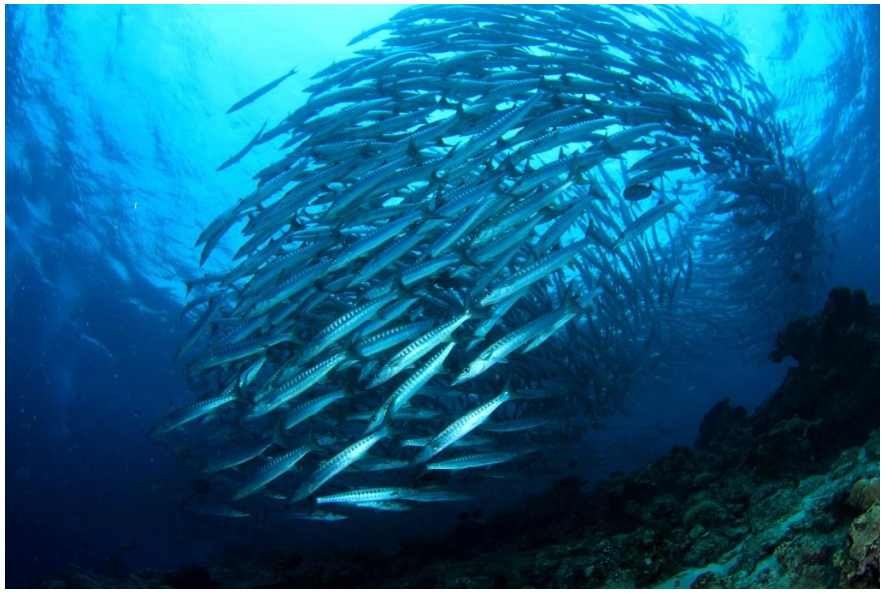


Figure 2. A. Ants in a pheromone trail between nest and food; **B.** an obstacle interrupts the trail; **C.** ants find two paths to go around the obstacle; **D.** a new pheromone trail is formed along the shorter path.

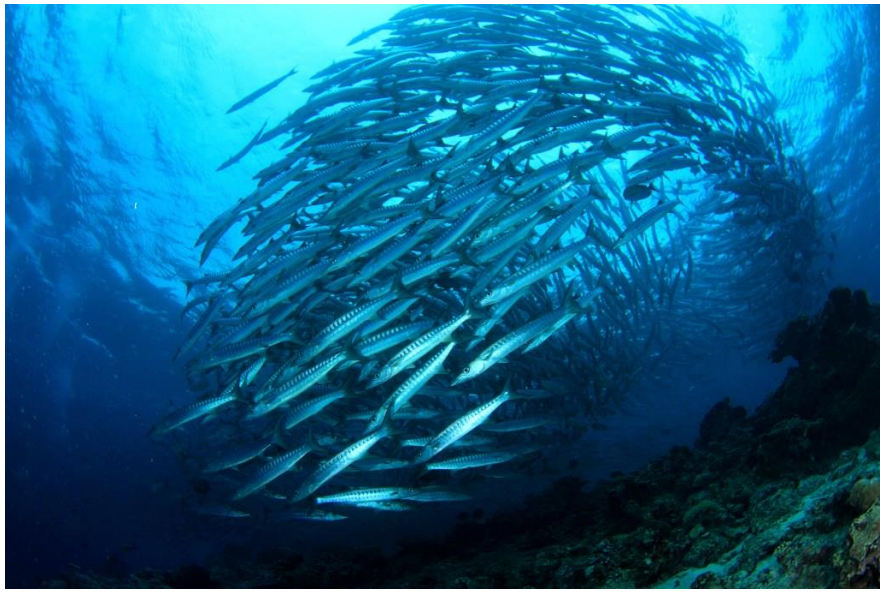
- As estratégias modelam o “comportamento social” observado em diversos tipos de colônias.
  - Inicialmente, os indivíduos se movimentam sem nenhuma orientação prévia;
  - Se aglomeram em colônias, até que um consegue encontrar o ninho ou a comida;
  - Este atrai os que estiverem mais próximos e a chance dos outros encontrarem aumenta consideravelmente.

Particle Swarm Optimization: Características Gerais



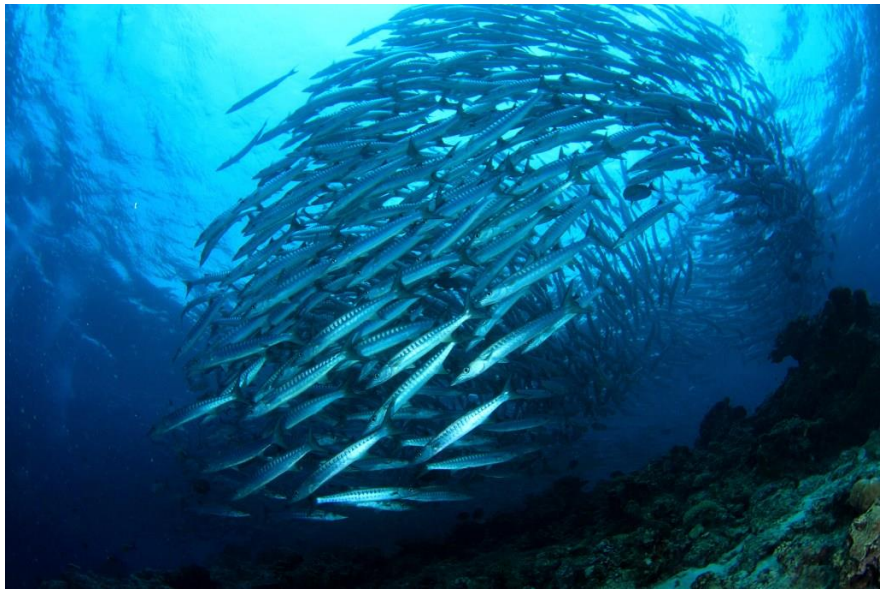
- Sempre baseados em duas variáveis básicas:
  - Posição
  - Velocidade
- Estratégia de busca da melhor solução (exemplo: pássaros):
  1. Inicialmente, os pássaros voam sem nenhuma orientação prévia;
  2. Se aglomeram em bandos, até que um consegue encontrar o ninho ou a comida;
  3. Este atrai os que estiverem mais próximos;
  4. A chance dos outros encontrarem aumenta consideravelmente.

Particle Swarm Optimization: Taxonomia Fundamental



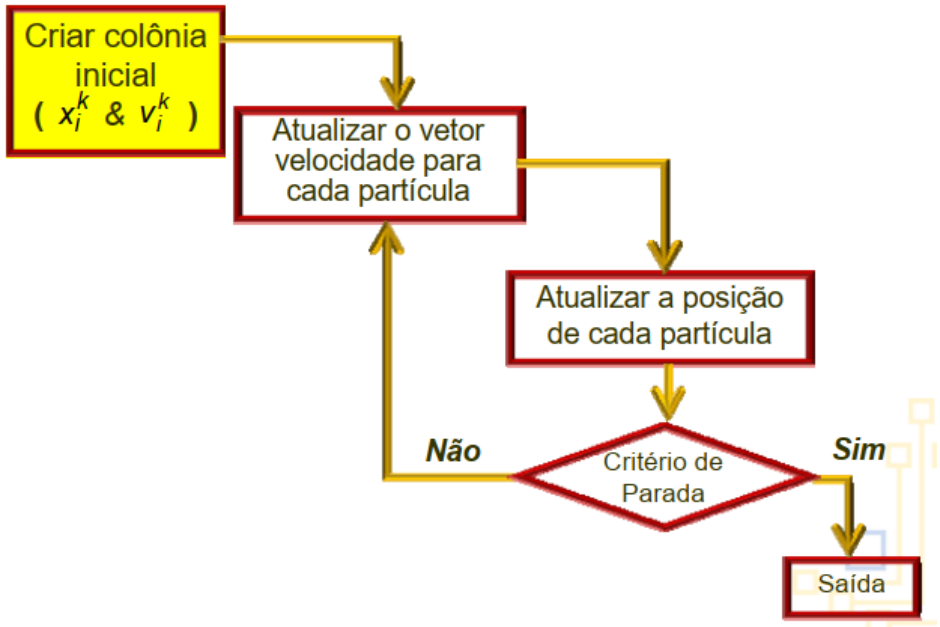
- Para o caso dos pássaros, estes são denominados de "partículas" do processo:
  - Durante a busca por alimento ou ninho, as "partículas" usam de suas experiências individuais e da experiência do próprio bando.
- Assim, o PSO é um algoritmo que possui um vetor de velocidades $\left(v_i^k\right)$ e outro de posição $\left(x_i^k\right)$:
  - v_i^k é a velocidade corrente da partícula i na iteração k.
  - x_i^k é a posição corrente da partícula i na iteração k.

Particle Swarm Optimization: Taxonomia Fundamental



- A posição de cada partícula é atualizada de acordo com a velocidade atual, levando-se em conta os seguintes aspectos:
  - Conhecimento adquirido pela “partícula” (p_{best}).
  - Conhecimentos adquirido pelo “bando” (g_{best}).

Particle Swarm Optimization: Estrutura Funcional

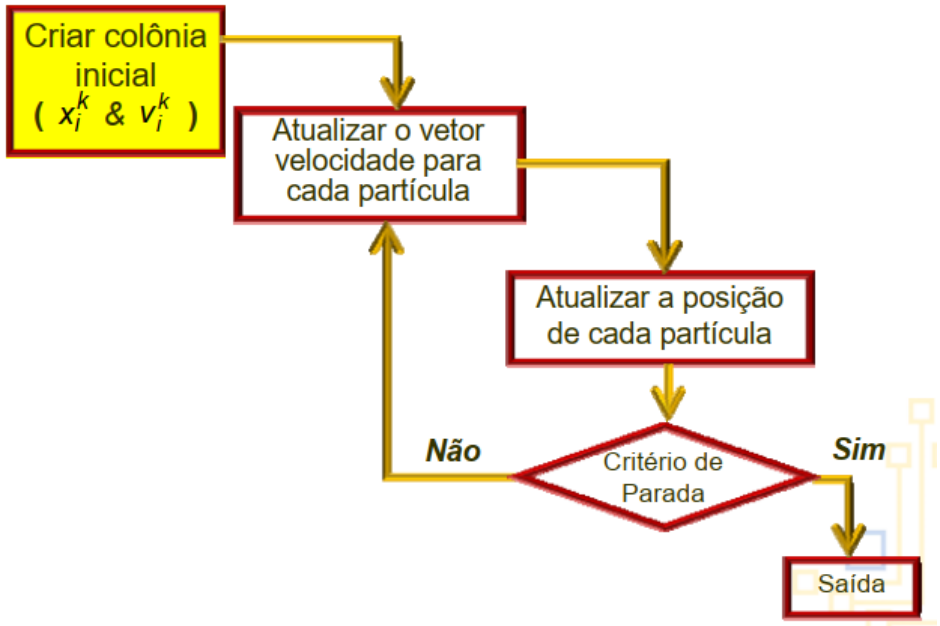


1. Criar a colônia inicial, com as posições e velocidades de cada partícula;
2. Atualizar a velocidade e posição de cada partícula;
3. Verificar o critério de parada;
4. Se o critério de parada foi satisfeito

   Então → encerrar processo.

   Senão → voltar ao passo 2.

Particle Swarm Optimization: Estrutura Funcional

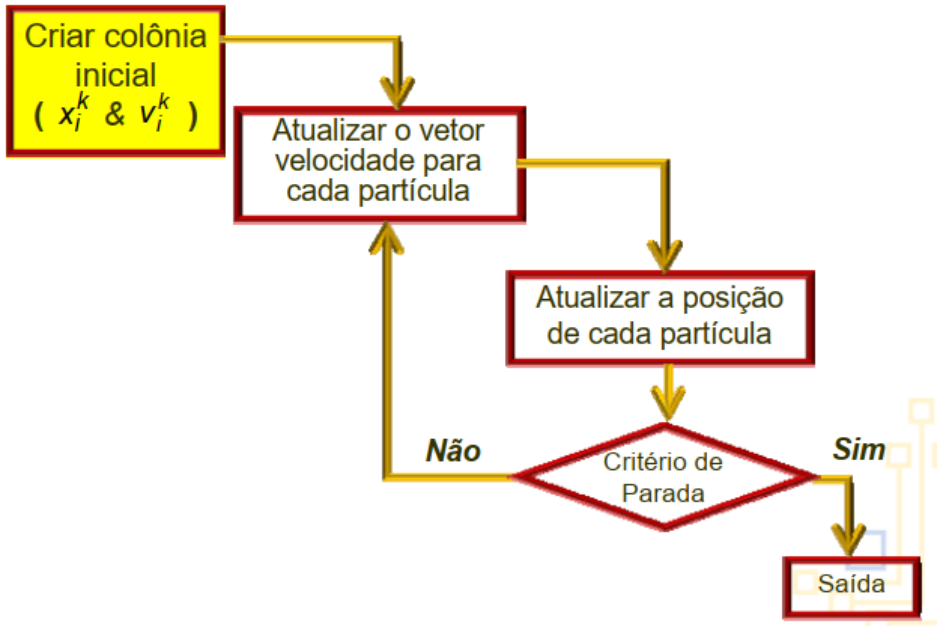


- A inicialização da população é normalmente obtida com as partículas dispostas aleatoriamente sobre o domínio de definição do problema;
- Cálculo da posição inicial:

$$x_i^{inicial} = x_{min} + rand_1 \cdot (x_{max} - x_{min})$$

- Onde:
  - $x_i^{inicial}$ é a posição inicial da partícula i;
  - x_{min} é o limite inferior do domínio de definição;
  - x_{max} é o limite superior do domínio de definição;
  - $rand_1$ é um número aleatório entre 0 e 1.

Particle Swarm Optimization: Estrutura Funcional



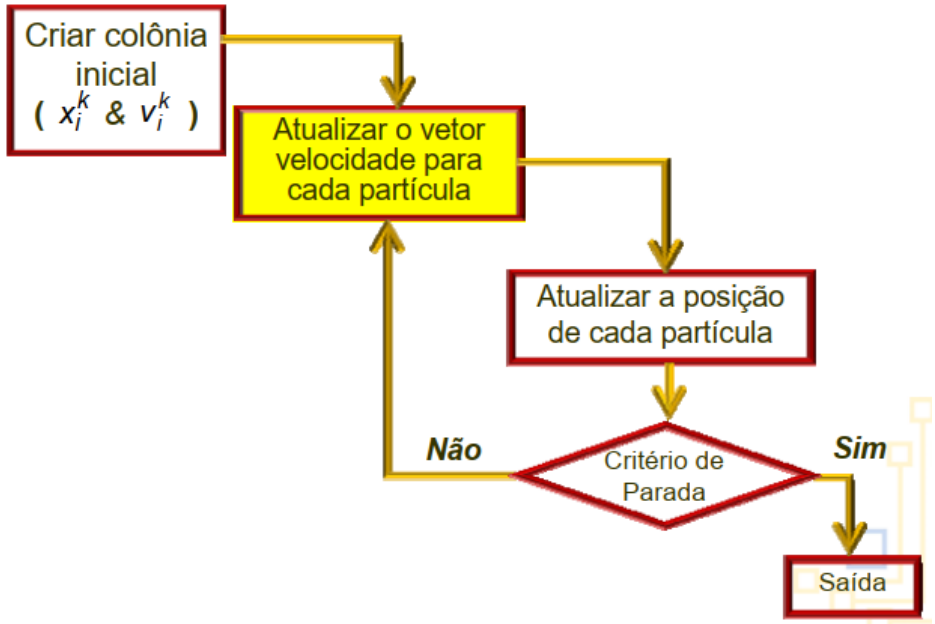
- Cada partícula possui também uma velocidade que foi inicializada aleatoriamente;
- Cálculo da velocidade inicial:

$$v_i^{inicial} = \frac{x_{min} + rand_2 \cdot (x_{max} - x_{min})}{\Delta t}$$

- Onde:
  - $v_i^{inicial}$ é a velocidade inicial da partícula i;
  - x_{min} é o limite inferior do domínio de definição;
  - x_{max} é o limite superior do domínio de definição;
  - $rand_2$ é um número aleatório entre 0 e 1.
  - Δt é o intervalo de tempo considerado.

Particle Swarm Optimization: Estrutura Funcional

Regra de Ajuste da velocidade



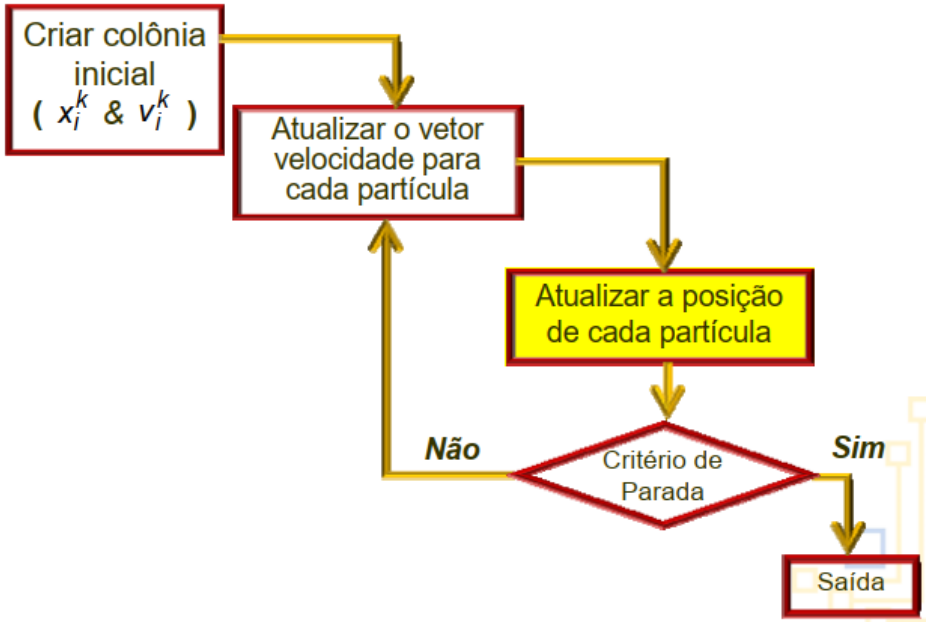
- O vetor de velocidades pode ser atualizado segundo a equação:

$$v_i^{k+1} = w_i \cdot v_i^k + c_1 \cdot rand \cdot \left(p_{best_i} - x_i^k\right) + c_2 \cdot rand \cdot \left(g_{best}^k - x_i^k\right)$$

- Onde:
  - v_i^k é a velocidade corrente da partícula i na iteração k ;
  - v_i^{k+1} é a velocidade atualizada da partícula i ;
  - x_i^k é a posição corrente da partícula i na iteração k ;
  - p_{best_i} é o melhor valor de retorno (custo) da partícula i ;
  - g_{best}^k é o melhor valor de retorno do bando na iteração k ;
  - $rand$ é um número aleatório entre 0 e 1;
  - c_1 e c_2 são coeficientes de ponderação (**parâmetros de confiança);**
  - w_i é uma função de ponderação para a velocidade da partícula i (**representa a sua inércia).**

Particle Swarm Optimization: Estrutura Funcional

Regra de Ajuste da posição



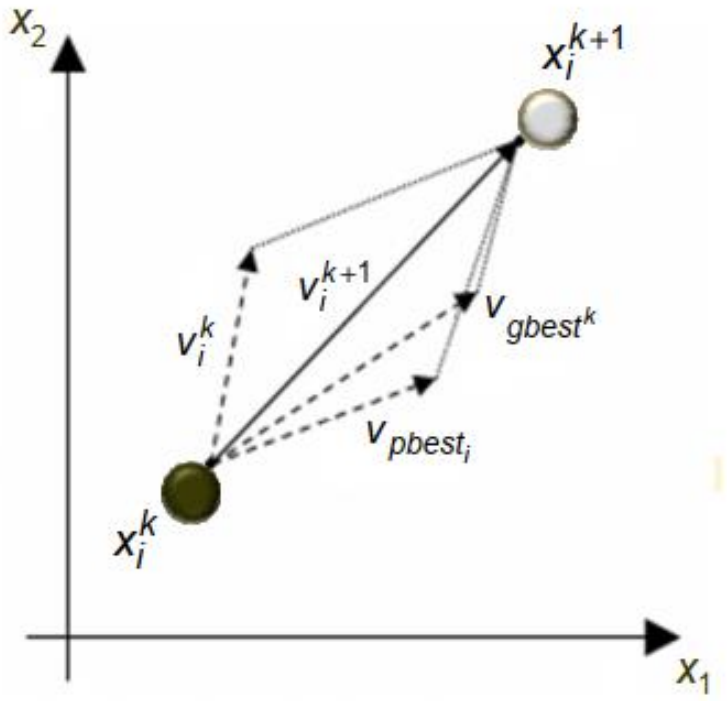
- A próxima posição das partículas é calculada usando-se a seguinte expressão:

$$x_i^{k+1} = x_i^k + v_i^{k+1} \cdot \Delta t$$

- Onde:
  - x_i^k é a posição corrente da partícula i na iteração k ;
  - x_i^{k+1} é a posição atualizada da partícula i na iteração k;
  - v_i^{k+1} é o vetor velocidade da partícula i na iteração k+1;
  - Δt é o intervalo de tempo considerado;

Particle Swarm Optimization: Estrutura Funcional

Interpretação Geométrica



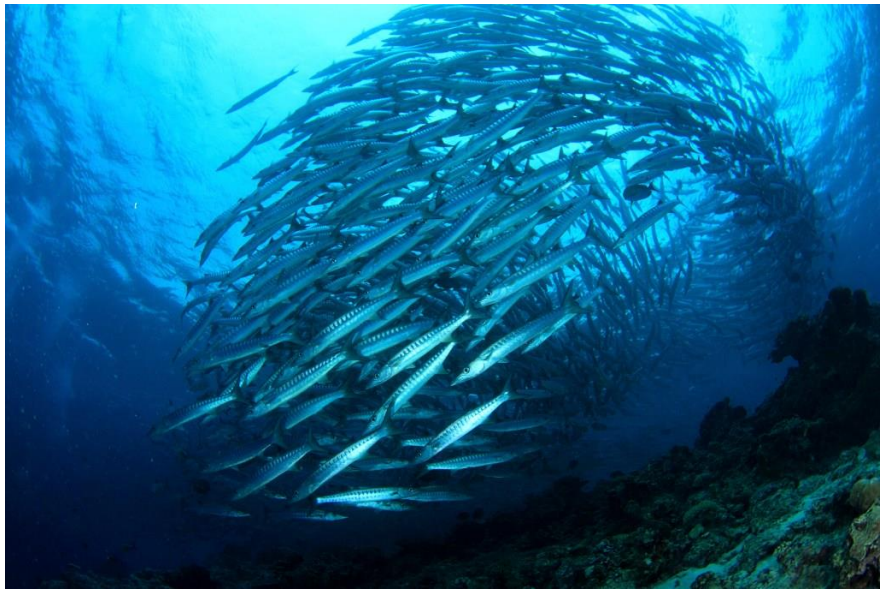
- Dadas as equações de atualização:

$$v_i^{k+1} = w_i \cdot v_i^k + c_1 \cdot rand \cdot \left(p_{best_i} - x_i^k\right) + c_2 \cdot rand \cdot \left(g_{best}^k - x_i^k\right)$$

$$x_i^{k+1} = x_i^k + v_i^{k+1} \cdot \Delta t$$

- Onde:
  - x_i^k é a posição corrente;
  - x_i^{k+1} é a posição atualizada;
  - v_i^{k+1} é a velocidade atualizada;
  - v_{pbest_i} é a velocidade em função de $p_{best(i)}$;
  - $g_{best\,i}$ é a velocidade em função de $g_{best(k)}$.

Particle Swarm Optimization: Estrutura Funcional



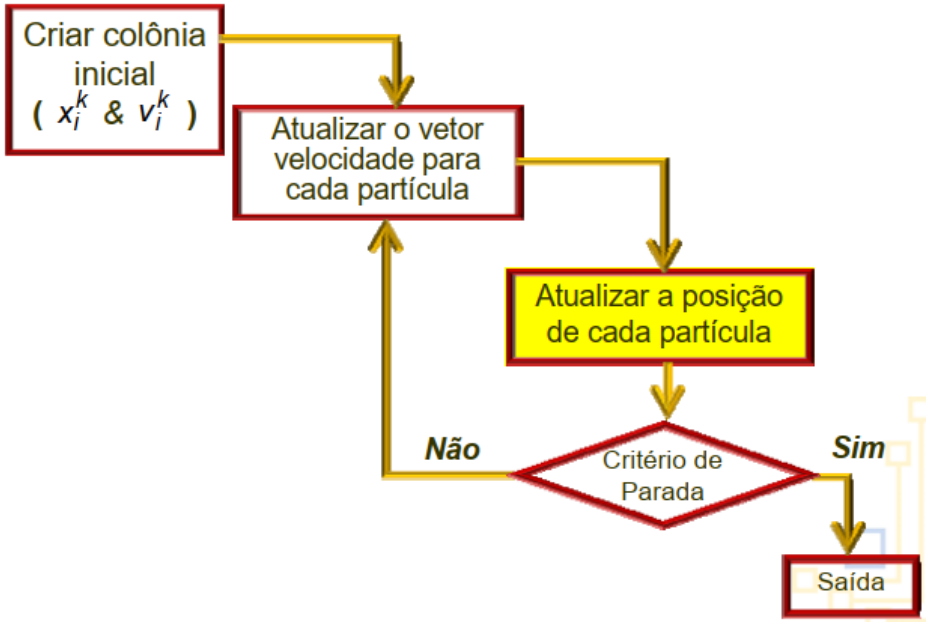
- **Velocidade máxima**: Um outro parâmetro presente é o v_{max} , vinculado a um procedimento de limitação de velocidade para as partículas, impede que o algoritmo desenvolva valores de velocidades não pertinentes ao problema;

$$-v_{max} \leq v_i^k \leq v_{max}$$

- Caso quaisquer das partículas do sistema desenvolva, em qualquer uma de suas dimensões, uma velocidade além de seus limites, este valor é automaticamente saturado, ou seja:

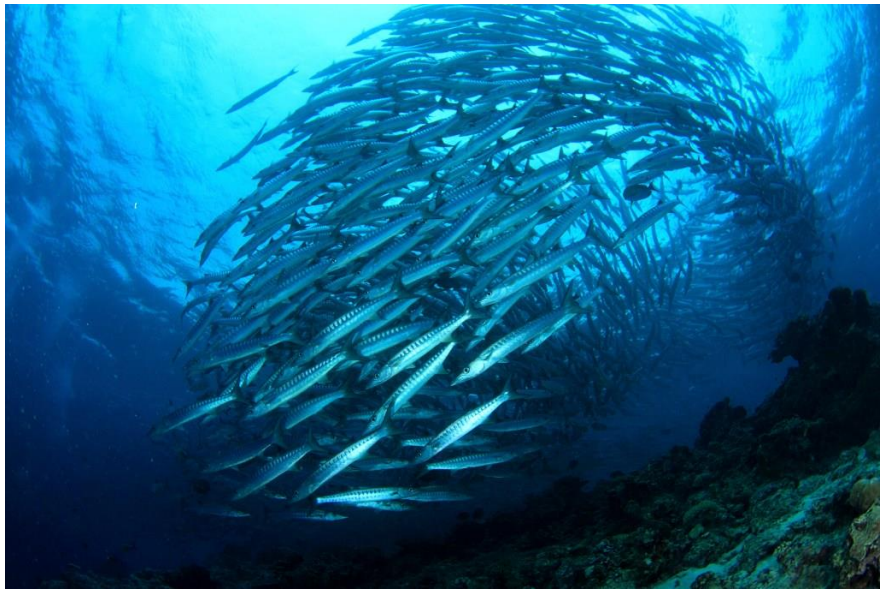
$$v_i^k \begin{cases} -v_{max}, se\ v_i^k \leq -v_{max} \\ v_{max}, se\ v_i^k \geq v_{max} \end{cases}$$

Particle Swarm Optimization: Critério de Parada



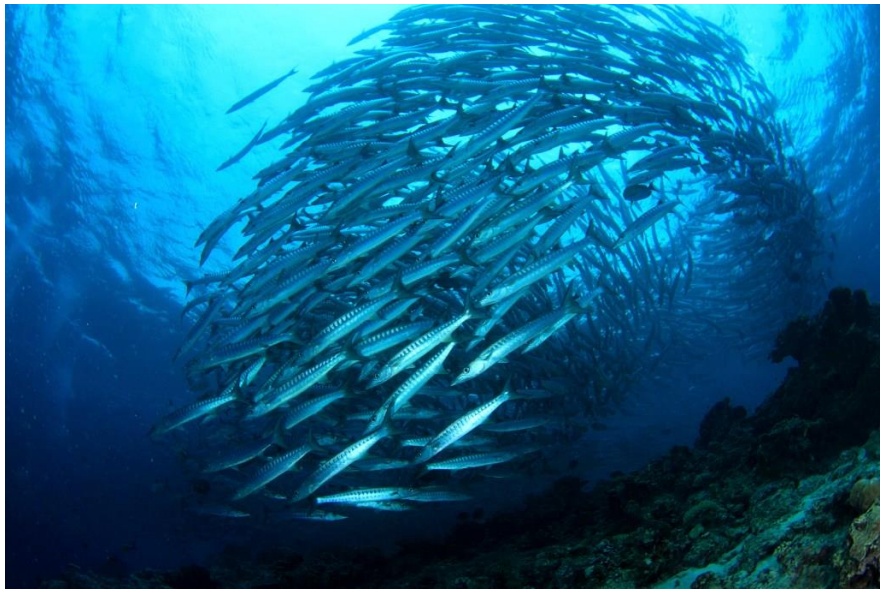
- São verificados o número máximo de iterações e o valor da melhor solução corrente;
- Caso algum deles seja satisfatório, o sistema é finalizado

Particle Swarm Optimization: Pseudo-código



1. Inicialize aleatoriamente as posições x_i^k das partículas.
2. Inicialize aleatoriamente as velocidades v_i^k das partículas.
3. Inicialize p_{best_i} e g_{best_k}
4. Loop Principal:

Particle Swarm Optimization: Pseudo-código



4. Para cada partícula i , faça as seguintes instruções:

   (Atualiza melhor posição da partícula atual)

   Se $x_i^k > p_{best_i}$ então

      Para cada dimensão d do problema, faça $p_{best_i} = x_i^k$

   Para cada partícula vizinha, faça **(atualiza melhor partícula)**:

      Se $p_{best_i} > g_{best_k}$ então $g_{best_k} = p_{best_i}$

   Para cada dimensão d do problema, faça

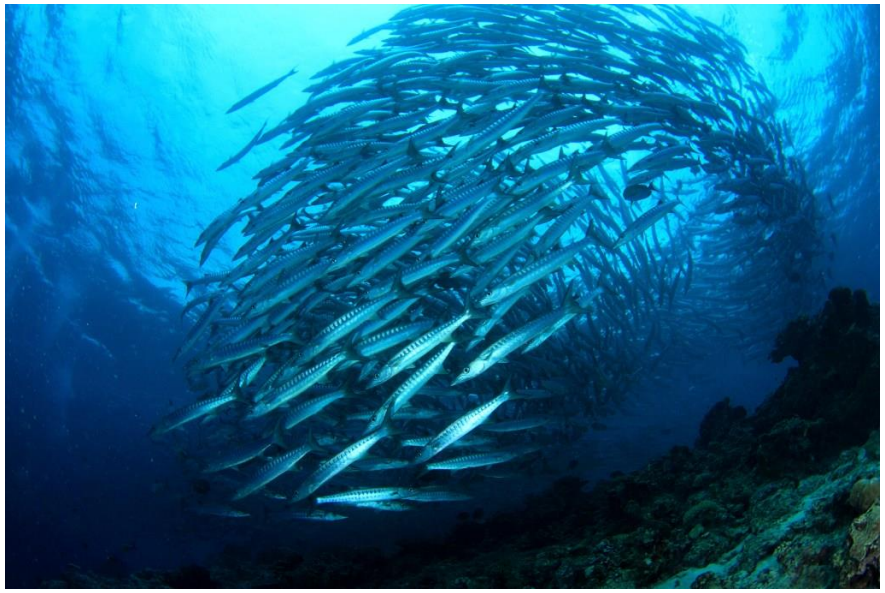
      Calcula v_i^{k+1} de acordo com a expressão dada

      Verifica-se a condição de v_{max}

5. Atualiza-se x_i^{k+1} de acordo com a expressão dada

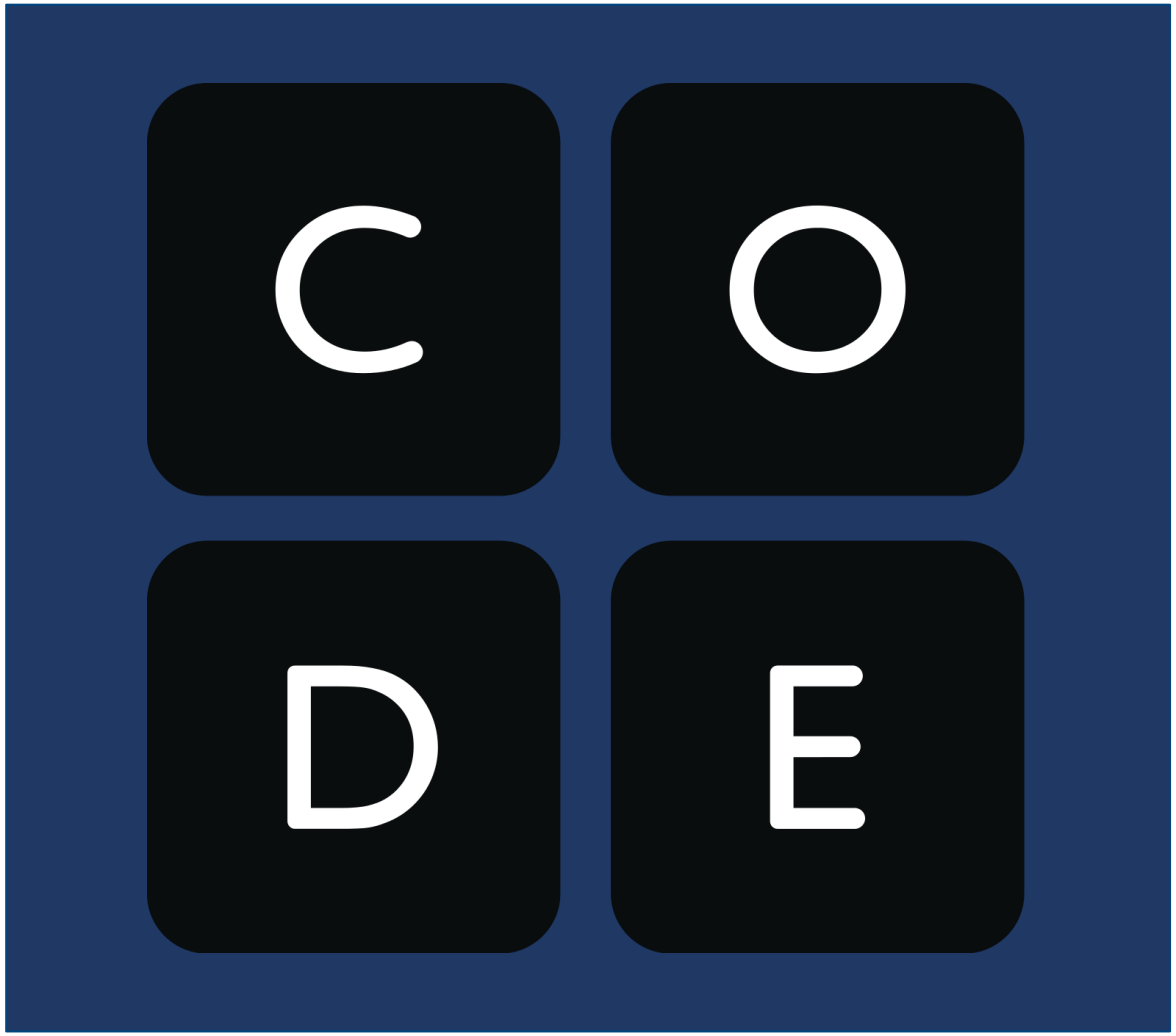
6. Verifica-se a condição de término e se não for satisfeita, volte ao passo 4.

Particle Swarm Optimization: Requisitos de Aplicação



- Conversão das possíveis soluções do problema a partir de sua representação por um vetor de posições e velocidades.
- Existência de um método para medir a qualidade (Função Objetivo) de uma solução potencial.

Challenge Time

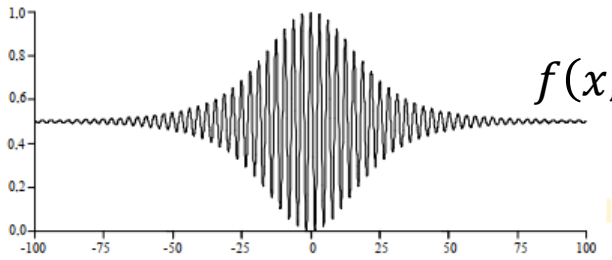




Laboratório 1: Função de Aptidão



- **Utilizando Jupyter e NumPy**: para a função de variável contínua apresentada, construa uma função para o cálculo do “custo” (função-custo).
- Plote a função abaixo.
- Notebook: facens_ml_Aula1_PSO_Lab01



$$f(x,y) = 0{,}5 - \frac{\left(sen\sqrt{x^2+y^2}\right)^2 - 0{,}5}{\left(1{,}0 + 0{,}001(x^2+y^2)\right)^2}$$

Laboratório 2: Parâmetros das Partículas



- **Utilizando Jupyter e NumPy**: Construa as funções que fazem os cálculos de pBest e gBest, para a função do exercício anterior.

- Notebook: **facens_ml_Aula1_PSO_Lab02**

- A) Construa **calcPBest**
- B) Construa a função auxiliar **calcAllPBest**
- C) Construa **calcGBest**

Laboratório 3: Gráfico de posição das Partículas



- **Utilizando Jupyter e NumPy**: Construa um algoritmo, que plota o gráfico do enxame de partículas para a função do exercício anterior.

- Notebook: **facens_ml_Aula1_PSO_Lab03**

Laboratório 4: Criação do enxame de Partículas



- **Utilizando Jupyter e NumPy**: Construa um algoritmo, que cria aleatoriamente o enxame de partículas para a função do exercício anterior.

- Notebook: **facens_ml_Aula1_PSO_Lab04**

Laboratório 5: Completar a função



- **Utilizando Jupyter e NumPy**: complete o algoritmo da função runEpoch, de acordo com o funcionamento esperado da mesma.

- Notebook: **facens_ml_Aula1_PSO_Lab05**

Laboratório 6: Completar o algoritmo PSO



- **Utilizando Jupyter e NumPy**: complete o algoritmo do loop principal para a execução do PSO.

- Notebook: **facens_ml_Aula1_PSO_Lab06**

Referências



- https://www.coursera.org/
- https://python.org.br/
- https://www.w3schools.com/python
- https://www.gnu.org/software/octave/
- https://octave-online.net/
- https://www.mathworks.com/products/matlab.html

Referências



- RUSSELL, STUART J.. Inteligência Artificial. 2ª ed. São Paulo: Campus - Elsevier, 2004.
- FERNANDES, Anita M. R.. Inteligência Artificial: noções gerais. 1ª ed. Florianópolis: Visual Books, 2008. Capítulo 1.
- SILVA, I.N. – Sistemas Inteligentes – Notas de Aula da disciplina de Sistemas Inteligentes – USP São Carlos, 2012.
- GOZALEZ, J.A.Q., ZAMPIROLLI, F.A. – Mineração de Dados e Regras de Associação. Notas de Aula da disciplina Sistemas Inteligentes. CMCC – UFABC, 2013.



Facens

AQUI TEM ENGENHARIA